scripts/02_step_tests.py

149 lines

```python
"""
02_step_tests.py
================
Open-loop step-test experiments used for dynamic model identification
(deliverable: "Open-loop step-test analysis").

Two independent experiments are run, both with the SAME noise-free plant
(noise is switched off here so the identified steady states and time
constants are not corrupted by measurement noise - exactly what an engineer
would do by averaging repeated real step tests):

* IDENTIFICATION set (config.STEP_SEQUENCE_ID) -> used by 03_identify_model.py
* VALIDATION set (config.STEP_SEQUENCE_VAL) -> held out, used only to
check the fitted model

Both up-steps and down-steps, small and large, from multiple starting points,
spanning the full 0-100 % range, each held for STEP_HOLD_HOURS (~4x the
slowest lag) so steady state is unambiguous.

Outputs
-------
data/step_test_identification.csv
data/step_test_validation.csv
figures/step_identification.png (4-panel response + choke overlay)
figures/step_validation.png
figures/step_gain_curve.png (steady-state gain vs operating point)
"""

from __future__ import annotations

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from config import DATA_DIR, FIG_DIR, STEP_HOLD_HOURS, STEP_SEQUENCE_ID, STEP_SEQUENCE_VAL
from well_simulator import WellSimulator
from plotting import COLOR_CHOKE, COLOR_Q, COLOR_WHP, COLOR_FLP, COLOR_BHP, style_axis, savefig


def run_step_sequence(sequence, hold_hours, noise, seed) -> pd.DataFrame:
"""Apply `sequence` of choke positions, each held for `hold_hours`."""
sim = WellSimulator(seed=seed, noise=noise)
sim.reset()
for u in sequence:
for _ in range(hold_hours):
sim.step(u)
df = sim.to_dataframe()
# tag which step index / commanded choke each row belongs to, for plotting
step_idx = np.minimum(np.asarray(df.index) // hold_hours, len(sequence) - 1)
df["step_index"] = step_idx
return df


def plot_step_test(df: pd.DataFrame, title: str, path: Path) -> None:
fig, axes = plt.subplots(5, 1, figsize=(11, 12), sharex=True)
t = df["time_h"]

axes[0].plot(t, df["choke_pct"], color=COLOR_CHOKE, lw=1.8, drawstyle="steps-post")
style_axis(axes[0], title=f"{title}: Choke Position", ylabel="Choke [%]")

axes[1].plot(t, df["Q"], color=COLOR_Q, lw=1.6)
style_axis(axes[1], title="Oil Flow Rate", ylabel="Q [bbl/hr]")
```

```python

axes[2].plot(t, df["WHP"], color=COLOR_WHP, lw=1.6)
style_axis(axes[2], title="Wellhead Pressure", ylabel="WHP [psi]")

axes[3].plot(t, df["FLP"], color=COLOR_FLP, lw=1.6)
style_axis(axes[3], title="Flowline Pressure", ylabel="FLP [psi]")

axes[4].plot(t, df["BHP"], color=COLOR_BHP, lw=1.6)
style_axis(axes[4], title="Bottom Hole Pressure", ylabel="BHP [psi]", xlabel="Time [h]")

fig.tight_layout()
savefig(fig, path)
print(f" wrote {path}")


def plot_gain_curve(sim: WellSimulator, path: Path) -> None:
"""Steady-state curve + local gain dQ/du vs operating point -> shows the
nonlinearity that motivates the gain-scheduled model."""
curve = sim.steady_state_curve(np.linspace(0, 100, 201))
dudq = np.gradient(curve["Q"], curve["choke_pct"])

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(curve["choke_pct"], curve["Q"], color=COLOR_Q, lw=2)
style_axis(axes[0], title="Steady-State Q vs Choke", xlabel="Choke [%]", ylabel="Q [bbl/hr]")

axes[1].plot(curve["choke_pct"], dudq, color="#7c3aed", lw=2)
style_axis(axes[1], title="Local Gain dQ/du (nonlinear!)", xlabel="Choke [%]",
ylabel="Gain [bbl/hr per %]")
fig.tight_layout()
savefig(fig, path)
print(f" wrote {path}")


def main():
print("Running IDENTIFICATION step-test sequence:", STEP_SEQUENCE_ID)
df_id = run_step_sequence(STEP_SEQUENCE_ID, STEP_HOLD_HOURS, noise=False, seed=1)
df_id.to_csv(DATA_DIR / "step_test_identification.csv", index=False)
plot_step_test(df_id, "Identification Step Test", FIG_DIR / "step_identification.png")

print("Running VALIDATION step-test sequence:", STEP_SEQUENCE_VAL)
df_val = run_step_sequence(STEP_SEQUENCE_VAL, STEP_HOLD_HOURS, noise=False, seed=2)
df_val.to_csv(DATA_DIR / "step_test_validation.csv", index=False)
plot_step_test(df_val, "Validation Step Test (held out)", FIG_DIR / "step_validation.png")

sim = WellSimulator(noise=False)
plot_gain_curve(sim, FIG_DIR / "step_gain_curve.png")

# ---- Written commentary printed to console + saved for the report ----
commentary = []
commentary.append("STEP-TEST OBSERVATIONS")
commentary.append("======================")
steps = STEP_SEQUENCE_ID
ends = df_id.groupby("step_index").last()
for i in range(1, len(steps)):
du = steps[i] - steps[i - 1]
dQ = ends.loc[i, "Q"] - ends.loc[i - 1, "Q"]
dWHP = ends.loc[i, "WHP"] - ends.loc[i - 1, "WHP"]
dFLP = ends.loc[i, "FLP"] - ends.loc[i - 1, "FLP"]
dBHP = ends.loc[i, "BHP"] - ends.loc[i - 1, "BHP"]
direction = "UP" if du > 0 else "DOWN"
commentary.append(
f"Step {i}: choke {steps[i-1]:>3}->{steps[i]:<3} ({direction:4s}, "
f"du={du:+.0f}%) -> dQ={dQ:+6.1f} bbl/hr dWHP={dWHP:+6.1f} psi "
f"dFLP={dFLP:+5.1f} psi dBHP={dBHP:+6.1f} psi "
f"gain dQ/du={dQ/du:+.2f}"
)
commentary.append("")
commentary.append(
"Coupling: every choke opening simultaneously RAISES Q, LOWERS WHP and "
"BHP (more drawdown), and RAISES FLP (more flow -> more line friction). "
```

```
"The gain dQ/du shrinks sharply as choke opens further (~3-4x higher "
"near 20% than near 80%), confirming the process is nonlinear across "
"its range -> motivates the gain-scheduled model in 03_identify_model.py."
)
text = "\n".join(commentary)
print("\n" + text)
(DATA_DIR / "step_test_commentary.txt").write_text(text)


if __name__ == "__main__":
main()

```